TASK 3

ADVANCED WEB, IDENTITY & SECURITY OPERATIONS ASSESSMENT

Modules 9–12 | Difficulty: Advanced

Security Assessment Report

Prepared for: Academic Assessment

Environment: Fictional SaaS Company — Controlled Lab Design

Prepared as a defensive, authorized-lab assessment. No real production systems were scanned or exploited.

Document Control & Scope

Purpose. This report designs an end-to-end security assessment and defensive program for a fictional SaaS company with a web frontend, REST APIs, identity services, database services, cloud-connected workloads and centralized logging.

Assessment approach. Architecture and controls are assessed through a documented design review and a proposed controlled-lab test plan. Findings are design-level risks to validate in a self-built or intentionally vulnerable lab; they are not claims that a real application was tested.

In scope: web and API security, authentication and authorization, sessions, input validation, TLS/PKI, IAM/RBAC/MFA, SOC/SIEM monitoring, Zero Trust, remediation and governance.

Out of scope: unauthorized systems, production exploitation, credential attacks, phishing, malware, denial-of-service and unnecessary collection of personal or sensitive data.

Executive Summary

The fictional SaaS environment should be protected through layered controls rather than relying on a perimeter firewall alone. The recommended design places a web application firewall (WAF) and API gateway in front of application services, centralizes identity through an identity provider (IdP), enforces authorization at every API operation, encrypts traffic and sensitive stored data, and forwards security-relevant telemetry to a SIEM. Administrative access should require phishing-resistant MFA where practical, managed devices, just-in-time elevation and auditable approvals.

The most important risks are broken object-level authorization (BOLA), weak session/token handling, excessive permissions, injection or unsafe input processing, secrets exposure, inadequate rate limits, insufficient audit logging and lateral movement from cloud workloads. The roadmap prioritizes high-impact access-control and identity fixes, followed by API hardening, telemetry and Zero Trust enforcement. All verification should occur in a lab or under written authorization.

Key recommendations

• Enforce deny-by-default authorization on every API endpoint and every object/tenant boundary.

• Use a standards-based IdP, MFA, short-lived sessions/tokens, secure cookies and safe token revocation.

• Validate inputs with allowlists and parameterized database queries; return generic client errors.

• Use TLS 1.2+ (prefer TLS 1.3), managed certificates, secure key storage and approved password hashing.

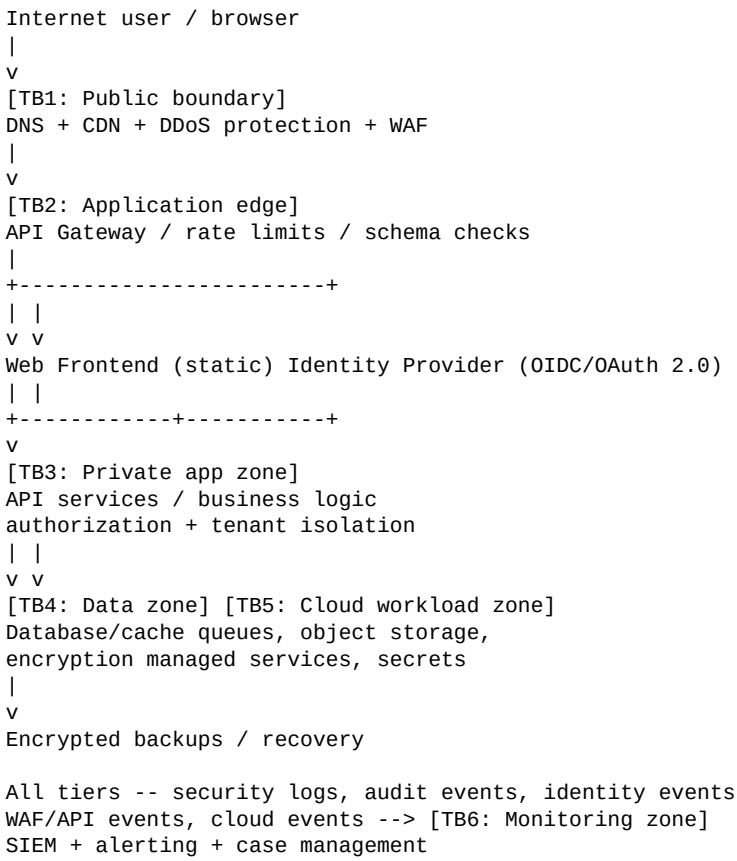
• Centralize logs with time synchronization, restricted access, retention controls and alert triage procedures.

• Apply Zero Trust: verify identity and device posture, grant minimum access, segment workloads and continuously evaluate risk.

1. Environment & Architecture

1.1 Logical architecture

The diagram below shows the proposed request path and trust boundaries. External users are untrusted; the edge tier is a controlled boundary; application services and data stores are isolated in private segments; the SIEM receives telemetry through a separate logging path.



1.2 Data flow and trust boundaries

Boundary	Data crossing it	Required controls
TB1 Public edge	Browser requests, public content, TLS	WAF, DDoS protection, TLS, secure headers, edge logging
TB2 API edge	HTTP requests, bearer tokens, JSON payloads	Gateway authentication checks, schema validation, quotas, request IDs
TB3 App zone	User identity/claims and business operations	Server-side authorization, tenant checks, service identity, secrets manager
TB4 Data zone	Queries, records, backups	Private networking, parameterized queries, encryption, least-privilege DB accounts
TB5 Cloud workloads	Events, objects, queues, service-to-service calls	Workload identities, scoped policies, private endpoints, secret rotation
TB6 Monitoring	Security events and audit records	Authenticated ingestion, time sync, access controls, integrity and retention

1.3 Sensitive data classification

• Restricted: passwords/password hashes, session tokens, API keys, signing keys, customer personal data and regulated records.

• Confidential: tenant data, business records, internal architecture, support tickets and security investigations.

• Internal: operational metrics, deployment metadata and non-sensitive configuration.

• Public: approved documentation, public marketing content and intentionally public API descriptions.

2. Threat Model

2.1 Assets, actors and attack surfaces

Category	Examples	Security objective
Assets	Customer/tenant records, identity credentials, tokens, source code, cloud secrets, audit logs, backups	Confidentiality, integrity, availability and accountability
Actors	Unauthenticated visitor, legitimate customer, malicious tenant, compromised employee, administrator, third-party integration	Assume both external and insider threats
Attack surfaces	Login/reset flows, REST endpoints, object IDs, uploads, browser storage, admin console, CI/CD, cloud APIs, dependencies	Reduce exposure and validate every boundary
Trust assumptions	IdP claims are valid; service identities are scoped; logs are trustworthy; tenants are isolated	Verify assumptions and detect failures

2.2 Threat scenarios (STRIDE-informed)

Threat	Example scenario	Mitigation / evidence
Spoofing	Stolen session or forged identity token	OIDC validation, MFA, short sessions, token audience/issuer checks
Tampering	Client alters tenant ID or price field	Server-side validation, object-level authorization, integrity checks
Repudiation	Admin changes a role without traceable evidence	Immutable-style audit pipeline, actor/target/time/outcome fields
Information disclosure	API returns another tenant's record	Tenant-scoped queries, response minimization, BOLA tests
Denial of service	Automated requests exhaust login/API capacity	Rate limits, quotas, caching, resource budgets and alerting
Elevation of privilege	User modifies role or accesses admin endpoint	Deny-by-default RBAC, server-side checks, privileged access reviews

2.3 Risk scoring method

Use a 5×5 qualitative scale. Likelihood and impact each range from 1 (low) to 5 (very high); risk score = likelihood × impact. Suggested bands: 1–4 Low, 5–9 Moderate, 10–16 High, 17–25 Critical. Scores below are initial design estimates and should be recalibrated using lab evidence and business context.

Risk register

ID	Risk	L×I	Rating	Treatment / owner
R1	BOLA or cross-tenant data exposure	4×5=20	Critical	Add per-object and tenant authorization; negative tests. API owner
R2	Account takeover from weak MFA/session controls	4×5=20	Critical	MFA, secure recovery, token rotation and revocation. IAM owner
R3	Injection or unsafe query construction	3×5=15	High	Parameterized queries, allowlist validation, SAST and lab tests. AppSec
R4	Excessive IAM/cloud permissions	4×4=16	High	Least privilege, role review, workload identities. Cloud owner
R5	Missing logs delay detection/response	4×4=16	High	Centralize logs, define alert coverage and test detections. SOC lead
R6	Secrets exposed in code/configuration	3×5=15	High	Secrets manager, scanning, rotation and access restrictions. DevOps
R7	API abuse / resource exhaustion	3×4=12	High	Per-user/IP/tenant quotas, payload limits and monitoring. API owner
R8	Weak TLS/key lifecycle or unencrypted backups	2×5=10	High	Managed certificates, KMS/HSM where appropriate, encrypted backups. Platform

3. Web & API Security Assessment

3.1 Assessment methodology

Use a self-built demo API or an intentionally vulnerable training application in an isolated lab. Establish written scope, test accounts and synthetic data. Capture minimal evidence: request/response excerpts with tokens and personal data removed, test timestamps, expected versus actual result, and a remediation note. Do not test third-party or production targets without explicit written authorization.

Recommended workflow

• Map endpoints and roles from the API specification; identify public, user, tenant-admin and platform-admin operations.

• Review code/configuration for authentication middleware, authorization checks, input validation, error handling, CORS, headers and secret handling.

• Run safe negative tests with synthetic records: unauthenticated request, wrong role, wrong tenant/object ID, malformed schema, oversized payload and repeated requests within agreed limits.

• Record each test as: ID, precondition, test action, expected behavior, observed behavior, evidence reference, severity and remediation.

• Retest fixes and record pass/fail; never include live credentials, tokens, real customer records or unnecessary sensitive information.

3.2 OWASP-focused checklist

Area	Verification question	Secure expected behavior
Broken access control / BOLA	Can a user access another user's or tenant's object by changing an ID?	403/404 without leaking object existence; server validates ownership/tenant
Authentication failures	Are login, recovery and MFA flows resistant to abuse?	MFA for sensitive access, safe recovery, throttling, no account enumeration
Injection	Are untrusted values concatenated into SQL/commands/templates?	Parameterized queries, context-aware encoding, allowlisted input
Cryptographic failures	Are sensitive values exposed in transit or storage?	Modern TLS, encrypted storage, approved password hashing, managed keys
Security misconfiguration	Are debug routes, default accounts or permissive CORS enabled?	Hardened config, minimal CORS origins, secure headers, no debug in production
Vulnerable components	Are dependencies outdated or unsupported?	Inventory/SBOM, patch SLAs, dependency scanning and controlled upgrades
Identification/authentication	Are tokens validated for signature, issuer, audience and expiry?	Complete validation; short expiry; rotation/revocation strategy
Integrity failures	Can deployment artifacts/configuration be altered without review?	Protected branches, signed/reviewed releases, CI/CD least privilege
Logging/monitoring failures	Are auth, access-denied and privileged actions visible?	Structured central logs, alerts, synchronized time and incident runbooks
SSRF / unsafe outbound requests	Can user input cause requests to internal metadata or private hosts?	Destination allowlists, egress filtering, block link-local/private targets as appropriate

3.3 API security controls

• Authentication: use a trusted IdP and standards-based OAuth 2.0/OIDC flows; validate token signature, issuer, audience, expiry and intended use.

• Authorization: check role, tenant and object permissions in the service layer on every request; never trust client-supplied role/tenant fields.

• Rate limiting: apply layered limits by IP, account, token, tenant and endpoint; use stricter limits for login, recovery, search and expensive operations.

• Validation: enforce content type, schema, size, type, range and allowlist constraints; reject unexpected fields where appropriate.

• Logging: capture request/correlation ID, timestamp, route, actor/service identity, tenant, outcome, latency and security decision without logging secrets.

• Errors: return consistent status codes and generic client messages; keep stack traces, SQL details and internal identifiers in restricted server logs.

• Transport and browser controls: HTTPS everywhere, HSTS where appropriate, secure headers, restrictive CORS and CSRF protection for cookie-authenticated state changes.

3.4 Finding templates (validate in lab)

The following are candidate findings based on common design failure modes, not asserted observations from a completed live test.

Finding	Severity (initial)	Evidence to collect	Recommended fix
F-01: Object-level authorization not consistently enforced	Critical if cross-tenant access is confirmed	Synthetic user A requests user B's lab object; redact identifiers	Centralize policy checks and add negative regression tests
F-02: Session cookies lack secure attributes	High if confirmed	Cookie attributes from lab response; no cookie values	Set Secure, HttpOnly, appropriate SameSite; rotate after privilege changes
F-03: API has no meaningful throttling	High if confirmed	Small, rate-limited test series and response headers	Apply quotas, burst limits and alerting; avoid load/DoS testing
F-04: Verbose errors disclose internals	Moderate/High	Synthetic malformed request and sanitized error response	Generic client error; detailed restricted server-side logs
F-05: Audit events omit actor/target/outcome	High	Example lab audit event schema	Add identity, target, decision, timestamp, correlation ID and outcome

4. Authentication, Authorization & Sessions

4.1 Authentication design

Use a central identity provider with OIDC for user sign-in and OAuth 2.0 authorization for API access. Prefer authorization-code flow with PKCE for browser-based public clients. Avoid embedding client secrets in browser/mobile code. Enforce MFA for all administrators and sensitive actions; prefer phishing-resistant security keys/passkeys where supported. Provide secure recovery with rate limits, notifications and risk checks.

4.2 Session and token controls

• Browser sessions: cookies should be Secure, HttpOnly and SameSite appropriately configured; use CSRF tokens or equivalent protections for state-changing cookie-authenticated requests.

• Use short-lived access tokens and protect refresh tokens; rotate refresh tokens and detect reuse where supported.

• Validate token signature, issuer, audience, expiry and relevant claims. Do not accept unsigned or algorithm-confused tokens.

• Regenerate session identifiers after login and privilege changes; invalidate sessions on logout, account recovery and suspected compromise.

• Do not store long-lived bearer tokens in localStorage when safer architecture choices are available; minimize browser-accessible secrets.

• Use generic login/recovery errors and rate-limit sensitive endpoints without creating a trivial denial-of-service path against legitimate users.

4.3 Authorization and tenant isolation

RBAC assigns permissions to roles; tenant and object policies provide additional context. Role checks alone are not enough: every data access must verify the actor’s relationship to the specific object and tenant. Use deny-by-default policies, centralized authorization helpers, explicit service-to-service permissions and regression tests for horizontal and vertical privilege escalation.

Example role matrix

Action	Customer user	Tenant admin	Support analyst	Platform admin
Read own tenant records	Allow (scoped)	Allow (scoped)	Ticket-approved, time-bound	Only if explicitly required
Manage tenant users	Deny	Allow within tenant	Deny by default	Controlled admin workflow
View security/audit events	Own activity only	Tenant-level subset	Approved support scope	Platform scope, audited
Change platform configuration	Deny	Deny	Deny	JIT elevation + approval
Export all tenant data	Deny by default	Policy-limited	Deny by default	Exceptional, approved, logged

5. Cryptography, TLS & PKI

5.1 TLS and certificates

Use TLS 1.2 or later and prefer TLS 1.3 where compatible. Disable obsolete protocol versions and weak cipher suites through managed platform configuration. Certificates bind a public key to a validated identity; clients validate the certificate chain to a trusted root, hostname and validity period. Automate certificate issuance, renewal and expiry monitoring. Use mutual TLS selectively for high-trust service-to-service paths where operationally appropriate.

5.2 PKI lifecycle

• Use an approved certificate authority and document issuance, renewal, revocation and ownership.

• Protect private keys with a managed key service or HSM for higher-impact keys; restrict export and access.

• Monitor certificate expiry, unexpected certificate changes and key-use anomalies.

• Separate development, test and production keys/certificates; never copy production private keys into a lab.

• Rotate compromised keys, revoke certificates where needed and document incident response steps.

5.3 Hashing, encryption and data protection

Use case	Recommended approach	Avoid
Passwords	Argon2id, scrypt or bcrypt with unique salts and tuned work factors	Plaintext, reversible encryption as password storage, fast general-purpose hashes alone
Data in transit	TLS 1.2+/1.3 with valid certificates	Plain HTTP for credentials or sensitive data
Sensitive data at rest	Strong standard encryption (e.g., AES-256) through managed storage/KMS controls	Hard-coded keys or keys stored beside ciphertext
Integrity verification	Cryptographic hashes such as SHA-256 for integrity use; keyed MAC/signatures where authenticity is required	Treating an unkeyed hash as proof of sender identity
Secrets/API keys	Secrets manager, scoped access, rotation, audit logs	Secrets in source code, tickets, logs or public repositories
Backups	Encryption, isolated access, tested restoration and retention policy	Unprotected snapshots or untested recovery

Hashing is one-way and suited to password verification or integrity checks; encryption is reversible with a key and protects confidentiality. Digital signatures support authenticity and integrity using asymmetric keys. PKI manages trust in public keys through certificates and certificate authorities. Key management, access control, rotation and recovery are as important as algorithm selection.

6. IAM, RBAC & MFA Design

6.1 Identity lifecycle

• Joiner: provision from an approved source, assign role from job function, require MFA and baseline device controls.

• Mover: reassess access when responsibilities change; remove old permissions before granting new elevated access.

• Leaver: disable identity promptly, revoke sessions/tokens, remove group memberships and rotate shared secrets if exposure is possible.

• Service identities: use workload identity or short-lived credentials, avoid shared human accounts, scope permissions to a single service and log use.

• Periodic review: managers and system owners certify privileged and sensitive access at least quarterly, with faster reviews after incidents or major role changes.

6.2 Proposed roles and guardrails

Role	Allowed scope	Required guardrails
Customer user	Own account and authorized tenant resources	MFA encouraged/required by risk; tenant isolation; session controls
Tenant admin	Users and settings within one tenant	MFA, audit trail, step-up auth for sensitive changes
Support analyst	Minimum customer support data needed for assigned case	Just-in-time access, approval, masking, session recording where lawful
Security analyst	SIEM events and incident cases	Read-only by default; elevated response actions separately approved
Platform administrator	Platform infrastructure and configuration	Phishing-resistant MFA, managed device, privileged access workstation, JIT access
Service workload	Specific API/database/queue permissions	Short-lived identity, no interactive login, scoped policy, rotation

6.3 MFA and privileged access

• Require MFA for all privileged roles and remote administrative access; use phishing-resistant factors where possible.

• Require step-up authentication for role changes, data exports, payment/security setting changes and key operations.

• Use time-bound privileged access with approval, justification and automatic expiry.

• Alert on MFA factor changes, repeated failures, new privileged role assignments and unusual administrative sign-ins.

• Maintain break-glass accounts only where necessary, with sealed credentials, monitored use and routine tests.

7. SOC/SIEM Architecture

7.1 Logical flow

```
Web/WAF + API Gateway + Identity Provider + App Services
+ Database/Audit + Cloud Control Plane + Endpoint/EDR
|
v
Secure log collectors / agents
parsing, timestamp normalization, enrichment
|
v
Central SIEM / searchable storage
correlation rules + threat intelligence context
|
+---------+---------+
v v
Alert queue Dashboards/reports
|
v
Analyst triage -> case/ticket -> containment
-> recovery -> lessons learned
```

7.2 Minimum event sources

#	Event source / event	Useful fields
1	Successful and failed authentication	User, source IP, device, time, MFA result, risk result
2	Password reset and MFA factor changes	Actor, target account, channel, outcome, correlation ID
3	Session/token creation, revocation and suspicious reuse	Subject, issuer, audience, expiry, client, result; never log raw tokens
4	Authorization denied / forbidden object access	Actor, object category, tenant, endpoint, decision, reason code
5	Privileged role or group changes	Initiator, target, old/new role, approval, time
6	WAF and API gateway detections	Rule ID, route, source, action, status, request ID
7	Rate-limit or quota violations	Account/tenant/IP key, route, count/window, action
8	Application validation and security errors	Endpoint, schema/rule ID, status, sanitized error category
9	Database audit events	DB identity, operation, target table/category, result; avoid sensitive row content
10	Cloud control-plane changes	Principal, resource, action, region, source, result
11	Secrets/key/certificate access or rotation	Principal, key/secret identifier, operation, success/failure
12	Endpoint/EDR and workload security alerts	Host/workload ID, alert type, severity, process/network metadata
13	Data export/download events	Actor, tenant, record count/size band, destination class, approval
14	Backup, restore and deletion events	Initiator, resource, outcome, retention/immutability status

8. SIEM Detection Use Cases & Analyst Playbooks

Thresholds are starting points for a lab and must be tuned to baseline behavior. Correlate by user, tenant, source, device and time. Avoid alerting on a single weak signal when a safe, multi-signal rule is possible.

Use case	Detection logic (example)	Analyst investigation
D1: Password spraying / repeated failures	Many failed sign-ins across several accounts from one source or distributed sources in a short window	Check source reputation, targeted accounts, success after failures, MFA events and device history; contain source/account if justified
D2: Impossible or unusual sign-in	New geography/device plus unusual time or risk score; treat geo-velocity as a signal, not proof	Confirm user activity, VPN/travel context, device posture and token/session history; revoke sessions if compromise is likely
D3: Privilege escalation	Role/group grant to privileged role, especially by unusual actor or outside change window	Verify change ticket and approval, inspect actor session and follow-on actions; remove unauthorized grant
D4: BOLA / cross-tenant probing	Repeated access-denied events for multiple object IDs/tenants or suspicious ID enumeration pattern	Inspect sanitized requests, affected endpoints and tenant boundaries; preserve minimal evidence and test in lab
D5: API abuse / rate-limit bypass	Request volume or error ratio exceeds per-tenant baseline; many costly routes or repeated 429/5xx patterns	Check client identity, route, request sizes, token/tenant, business impact; apply scoped throttling and investigate automation
D6: Suspicious data export	Large or atypical export/download, unusual destination or activity outside normal role pattern	Validate business reason and approval; review actor, records/size, session risk and egress logs; pause export if authorized procedure permits
D7: Secret/key misuse	Access to secrets by a new principal, unusual time, high-volume reads or unexpected key export attempt	Review IAM policy and workload deployment, correlate cloud changes; rotate/revoke if exposure is credible
D8: Logging/telemetry tampering	Logging disabled, agent stops reporting or audit configuration changed	Check deployment/change history and collector health; restore telemetry and treat unexplained suppression as incident
D9: Repeated validation/injection signals	WAF/API validation events across multiple endpoints or payload classes	Review sanitized payload fingerprints and target routes; confirm whether requests were blocked and whether app errors followed
D10: Backup deletion or recovery disruption	Backup deletion, retention change or restore failure by unusual identity	Validate approved change, check linked cloud actions and recent account risk; protect remaining recovery copies

8.1 Alert severity and response

• Critical: credible cross-tenant data access, active privileged compromise, key exposure or destructive backup activity. Page on-call; contain according to approved playbook.

• High: suspicious privilege change, likely account takeover, sustained API abuse or material telemetry tampering. Triage promptly and preserve evidence.

• Medium: repeated blocked probes, unusual sign-in without corroboration, isolated validation events. Enrich and correlate before escalation.

• Low/informational: expected policy denials or low-confidence anomalies. Retain for trend analysis and tune noisy rules.

8.2 Analyst workflow

Validate alert fidelity; establish timeline; identify affected identities, devices, tenants and services; correlate authentication, gateway, application and cloud events; determine scope and business impact; preserve only relevant evidence; follow approved containment authority; document decisions and timestamps; recover safely; and tune the detection after closure. Do not place secrets or raw tokens in case notes.

9. Zero Trust Security Design

Zero Trust assumes no user, device, workload or network location is trusted automatically. Access is explicitly authorized, least-privileged and continuously evaluated against identity, device posture, context and resource sensitivity.

Dimension	Design decision	Verification
Users	Central IdP, MFA, risk-based/step-up auth, lifecycle automation	Review sign-in and access-review records
Devices	Managed-device posture, patch/EDR status, disk encryption and compliance checks	Deny or restrict noncompliant devices for sensitive actions
Applications	Service identity, per-request authorization, secure API gateway, workload segmentation	Test service identity scopes and cross-tenant negative cases
Network	Private service paths, segmented data tier, controlled egress, no implicit trust based on subnet	Review flow logs and confirm direct public database access is blocked
Data	Classification, least-privilege access, encryption, export controls and audit trails	Verify access policies, encryption settings and export alerts
Operations	Central logging, continuous monitoring, time-bound privilege, incident playbooks	Run tabletop exercises and validate alert-to-case flow

Implementation principles

• Authenticate and authorize every request, including internal service calls.

• Grant access to a specific resource for a defined purpose; avoid broad network-based trust.

• Use device and workload identity posture as part of access decisions where supported.

• Segment production, staging, development and security monitoring environments.

• Continuously evaluate risk signals and revoke or reduce access when compromise is suspected.

• Measure outcomes: MFA coverage, privileged access review completion, patch compliance, logging coverage and mean time to detect/respond.

10. Remediation Roadmap

Priorities are proposed for the fictional environment. Confirm owners, dependencies and dates with the organization before execution.

Phase	Timeframe	Actions	Exit criteria
0 — Immediate	0–2 weeks	Confirm inventory/scope; fix confirmed critical authorization issues; require admin MFA; protect exposed secrets; verify backups	Critical risks assigned; urgent controls enabled; evidence recorded
1 — Foundation	2–6 weeks	Centralize IdP; standardize token/session validation; enforce schema validation; add API quotas; harden TLS and cookies	Security tests pass for key auth/access-control cases
2 — Visibility	4–8 weeks	Ingest required event sources; normalize fields; deploy first 5–10 detections; define triage ownership	Logs arrive reliably; test alerts create actionable cases
3 — Zero Trust	6–12 weeks	Managed-device policy, private service paths, workload identities, JIT privilege and access reviews	Privileged access is time-bound; sensitive paths are segmented
4 — Assurance	Quarterly/ongoing	Dependency and configuration reviews, penetration testing under authorization, incident tabletop, restore tests and metrics	Findings tracked to closure; playbooks and controls retested

11. Metrics & Governance

Track a small set of measurable indicators so leadership can determine whether risk is decreasing. Metrics should be reviewed monthly by security and system owners.

Metric	Suggested target / interpretation
MFA coverage	100% of privileged accounts; track all-user adoption and exceptions
Access review completion	100% of privileged and sensitive roles reviewed each quarter
Critical finding remediation	Target agreed SLA based on severity and exposure; report overdue exceptions
API authorization test coverage	All sensitive endpoints have positive and negative authorization tests
Log source coverage	All priority identity, edge, app, cloud and data sources monitored
Detection validation	Each high-priority rule has a documented test, owner and tuning date
Incident response	Measure time to acknowledge, scope, contain and recover; do not optimize speed at the expense of evidence
Backup recovery	Regular successful restore tests with documented recovery objectives

12. Authorized Lab Evidence Plan

This report intentionally does not claim that hands-on tests were executed. Complete the following evidence items in a self-built or intentionally vulnerable lab, using synthetic accounts and data. Include screenshots or sanitized text outputs in the GitHub repository, not live secrets.

Evidence ID	Lab activity	Evidence to include	Safety note
E-01	Architecture and data-flow review	Diagram showing trust boundaries and sensitive flows	Use fictional hostnames and sample data
E-02	Authentication/session review	Redacted response headers and test notes for cookie flags/session behavior	Never show real cookies or tokens
E-03	Authorization negative tests	Two synthetic users/tenants; expected denial for cross-tenant object access	Do not target production or real customer data
E-04	Input validation/error handling	Malformed but safe JSON/schema test and sanitized response	No destructive payloads or stress tests

Evidence ID	Lab activity	Evidence to include	Safety note
E-05	Rate-limit verification	Small, controlled request series with configured threshold	Do not conduct denial-of-service testing
E-06	TLS/certificate inspection	Lab certificate chain and protocol/configuration summary	No private keys in evidence
E-07	SIEM detection test	Synthetic event or lab alert with rule name, timestamp and analyst notes	Use sanitized identities and event data
E-08	IAM/MFA review	Role matrix, MFA policy and sample access-review record	No passwords, recovery codes or secrets

For every test, record: test ID; date/time; lab version; tester role; precondition; action; expected result; observed result; evidence filename; severity; remediation; retest result. Mark unexecuted tests as “Planned” rather than “Passed.”

13. GitHub Repository Structure & README

The submission instructions require only the GitHub repository link in the LMS. Create a public repository only if your institution permits public coursework; otherwise use the required private visibility and ensure the evaluator has access.

Recommended repository layout

```
task-3-advanced-web-identity-security/
■■■ README.md
■■■ report/
■ ■■■ Task_3_Assessment_Report.pdf
■■■ diagrams/
■ ■■■ application-architecture.png
■ ■■■ application-architecture.mmd
■ ■■■ soc-siem-architecture.mmd
■■■ threat-model/
■ ■■■ assets-actors-attack-surfaces.md
■ ■■■ risk-register.csv
■■■ web-api-assessment/
■ ■■■ checklist.md
■ ■■■ findings.md
■ ■■■ lab-evidence/
■ ■■■ README.md
■■■ identity-cryptography/
■ ■■■ iam-rbac-mfa.md
■ ■■■ tls-pki-data-protection.md
■■■ soc-siem/
■ ■■■ event-catalog.md
■ ■■■ detection-use-cases.md
■ ■■■ analyst-playbooks.md
■■■ zero-trust/
■ ■■■ design.md
■■■ remediation/
■■■ roadmap.md
```

README.md starter content

Use the following content as the repository landing page and adjust paths to match the files you actually upload.

```
# Task 3 — Advanced Web, Identity & Security Operations Assessment

## Overview
Defensive security assessment and security-program design for a fictional SaaS environment. All
testing is limited to an authorized, isolated lab and synthetic data.

## Scope
- Application architecture, data flows and trust boundaries
- Threat model and risk register
- OWASP-focused web/API assessment plan and lab evidence
- TLS, hashing, certificates, PKI and data protection
- IAM, RBAC, MFA and session controls
- SOC/SIEM event sources and detection use cases
- Zero Trust design and remediation roadmap

## Repository map
- `report/` — final assessment report
- `diagrams/` — architecture and SIEM diagrams
- `threat-model/` — assets, actors, threats and risks
- `web-api-assessment/` — checklist, findings and sanitized lab evidence
- `identity-cryptography/` — IAM and cryptography design
- `soc-siem/` — event catalog, detections and analyst playbooks
- `zero-trust/` — access and segmentation design
- `remediation/` — phased improvement roadmap

## Ethics and limitations
No production system was scanned or exploited for this report. Candidate findings are design
risks to validate; they are not represented as observed vulnerabilities unless supported by
attached lab evidence. No credentials, live tokens, private keys or real customer data should be
committed.

## How to review
Start with the PDF in `report/`, then review the diagrams, risk register, security controls and
lab evidence index. Lab tests should be marked Planned, Passed or Failed accurately.
```

14. Conclusion

A defensible SaaS security program combines secure application design, identity-centric access control, cryptographic protection, meaningful telemetry and rehearsed response. The highest-value first steps are to prove tenant/object authorization, strengthen authentication and session handling, reduce excessive permissions, protect secrets and ensure that key events reach the SOC. Zero Trust extends these controls across users, devices, applications, workloads and data. The organization should validate each control in an authorized lab, track remediation to closure and repeat tests after significant changes.

Reference framework pointers

• OWASP Top 10 for Web Application Security and OWASP API Security Top 10 — use the current official editions when mapping findings.

• OWASP Application Security Verification Standard (ASVS) — use as a structured verification checklist.

• NIST Cybersecurity Framework (CSF) 2.0 — governance, identify, protect, detect, respond and recover outcomes.

• NIST SP 800-63 — digital identity guidance; select controls appropriate to the system and risk.

• NIST SP 800-207 — Zero Trust Architecture concepts.

• RFC 8446 (TLS 1.3), RFC 6749 (OAuth 2.0), and OpenID Connect Core — protocol design references.

Submission checklist

• Upload the completed PDF and supporting Markdown/diagrams/evidence to the GitHub repository.

• Replace any placeholder evidence with actual sanitized lab evidence, or clearly label tests as Planned.

• Check that the PDF opens and all diagrams/tables are readable.

• Ensure no secrets, tokens, private keys, personal information or real customer data are committed.

- Copy the repository URL and submit only that link through the LMS, as instructed.